Gateway Resource Sizing

An overview about gateway resource sizing.

Overview

Resource recommendations for a Gateway instance are based on traffic, the deployment context, and expected usage.

The following matrix defines the most common use cases for an APIM Gateway and considers both the expected global throughput and the number of APIs that will be deployed.

Gateway size	Number of APIs	Throughput	Usage
Small	1 - 20	~200 req/s	Development, test, or small production environment that is not used intensively but may sometimes encounter peaks in traffic.
Medium	20 - 200	~1000 req/s	Real production environment that can handle considerable throughput.
Large	200+	5000+ req/s	Mission-critical environment such as a centralized enterprise Gateway that must handle a very high throughput.

Sizing recommendations

Sizing your Gateway instances

The Gravitee Gateway supports both container-based (cloud) and VM-based deployments.

Based on the above matrix summarizing the different use cases, we recommend using the minimum resource allocations shown in the tables below.

> These are informative estimates only and you should adjust allocations as needed.

Cloud-based deployments

Gateway size	CPU	System memory	Gateway memory
Small	500 millicore	512m	128m
Medium	750 millicore	768m	256m
Large	1000 millicore	1024m	512m

For a cloud-based architecture such as Kubernetes, adapt the CPU and memory of your pods depending on your requirements. For low latency, consider increasing CPU limits. For optimized payload transformation, consider increasing memory.

Container-based deployments are characterized by resource constraints, so instead of increasing your resources, we recommend adjusting your minimum and maximum number of replicas.

VM-based deployments

Gateway size	CPU	System memory	Gateway memory	Disk space
Small	1 core	1024m	256m	20 GB
Medium	2 cores	1536m	512m	20 GB
Large	4 cores	2048m	1024m	20 GB

VM-based deployments are resource-intensive and require more memory and CPU than container-based deployments.

Node sizing recommendations

The following table shows baseline hardware recommendations for a self-hosted deployment.

Component	vCPU	RAM (GB)	Disk (GB)
Dev Portal + REST API (Dev Portal only)	1	2	20
Console + REST API (Console only)	1	2	20
Dev Portal + Console + REST API	2	4	20
API Gateway instance Production best practice (HA) is 2 nodes.	0.25 - 4	512 MB - 8	20
Alert Engine instance Production best practice (HA) is 2 nodes	0.25 - 4	512 MB - 8	20
Analytics DB instance (Elasticsearch) Production best practice is 3 nodes ↗. Official hardware recommendations ↗.	1 - 8	2 - 8 or more	20 + 0.5 per million requests for default metrics
Config DB instance (MongoDB or JDBC DB) Production best practice is 3 nodes ↗	1	2	30
Rate Limit DB instance (Redis) Production best practice is 3 nodes ↗	2	4	20

Gravitee JVM memory sizing

You can specify the JVM memory sizing for each of the Gravitee nodes.

> - `GIO_MIN_MEM` is the same as `Xms` and `GIO_MAX_MEM` is the same as `Xmx`.
> - To avoid resizing during normal JVM operations, set the same value for both the `GIO_MIN_MEM` and the `GIO_MAX_MEM`.
> - AI-powered policies such as **AI Prompt Guard Rails** and **PII Filtering** load ONNX classification models into the Gateway's Java heap on the first request to an API that uses the resource. Without sufficient heap, the Gateway throws an `OutOfMemoryError` when the model loads. Size the heap based on the selected model and the number of APIs that use the resource. Available text classification models range from 4.39M parameters (BERT Tiny) up to approximately 300M parameters (Detoxify ONNX, and Llama Prompt Guard 86M in ONNX F32 representation). For per-model parameter counts and footprints, see AI Model Text Classification - Model Reference and Performance Metrics. The resource downloads model files to `$GRAVITEE_HOME/models` on first use, so the user running the Gateway needs write permission on that directory. The ONNX Runtime isn't supported on Alpine Linux. Use the Debian-based image (`graviteeio/apim-gateway:<version>-debian`).

Docker Compose

To configure JVM memory sizing with `docker compose`, complete the following steps:

1. In your `docker-compose.yml` file, navigate to the Gravitee component that you want to configure. For example, `gateway`.
2. In the `environment` section, add the `GIO_MIN_MEM` and the `GIO_MAX_MEM` lines with the value of the JVM heap size. Ensure that both these values are the same to avoid resizing during normal operations.

Here is an example configuration of the JVM for the Gravitee API Gateway.

docker-compose.yml

```
services:
  gateway:
    ...
    environment:
      - GIO_MIN_MEM=512m
      - GIO_MAX_MEM=512m
      ...
```

Note: During bootstrap, which occurs when the Gravitee component starts up, the `GIO_MIN_MEM` and `GIO_MAX_MEM` variables are injected into the `JAVA_OPTS`.

2. Run `docker compose up -d` to restart your containers with this new configuration.

Kubernetes (Helm)

When deploying containers within Kubernetes, it is typical to configure the JVM and resources at the same time. Best practice is to configure the JVM to be 70% of the defined resources. If you define `resources.limits.memory: 1024Mi` and define `resources.requests.memory:1024Mi`, then `GIO_MIN_MEM` and `GIO_MAX_MEM` should be `716m`.

> We recommend that you set the same value for `resources.limits.memory` and `resources.requests.memory`

To configure resources and JVM memory sizing with Kubernetes, complete the following steps:

1. In your `values.yaml` file, navigate to the Gravitee component that you want to configure. For example, `gateway`.
2. In the `env` section, add the following lines:

```
    ...
    env:
      - name: GIO_MIN_MEM
        value: <value>m
      - name: GIO_MAX_MEM
        value: <value>m
      ...
```

- Replace `<value>` with the value of your heap size. To avoid resizing during normal operations, ensure that this value is the same for the `GIO_MIN_MEM` and the `GIO_MAX_MEM`.

Here is an example of configuring resources and JVM of the API Gateway:

values.yaml

```
api-management:
  gateway:
    ...
    resources:
      limits:
        cpu: 1
        memory: 1024Mi
      requests:
        cpu: 500m
        memory: 1024Mi
    ...
    env:
      - name: GIO_MIN_MEM
        value: 1152m
      - name: GIO_MAX_MEM
        value: 1152m
      ...
```

Note: During bootstrap, which occurs when the Gravitee component starts up, the `GIO_MIN_MEM` and `GIO_MAX_MEM` variables are injected into the `JAVA_OPTS`.

2. To apply the updated configuration, redeploy the values.yaml file with your specific command `helm upgrade [release] [chart] -f values.yml`. For example, `helm upgrade gravitee-apim graviteeio/apim -f values.yml`

Sizing considerations

Capacity planning

Effective capacity planning relies on the specifics and optimization of storage, memory, and CPU.

Storage

Storage concerns reside at the analytics database level and depend on:

- Architecture requirements (redundancy, backups)
- API configurations (i.e., are advanced logs activated on requests and responses payloads)
- API rate (RPS: Requests Per Second)
- API payload sizes

To avoid generating excessive data and reducing Gateway capacity, refrain from activating the advanced logs on all API requests and responses.

For example, if you have activated the advanced logs on requests and responses with an average (requests + responses) payload size of 10kB and at 10 RPS, then retaining the logs for 6 months will require 1.5 TB of storage.

Request metrics are stored separately from advanced logs, in the request metrics index (`v4-metrics`). Each proxied request consumes an average of 180 bytes in this index, independent of payload size, because metrics documents don't carry the request or response body. Size the request metrics index on its own, since it grows at a very different rate from the logs. For example, at 100 RPS, retaining request metrics for 6 months requires approximately 280 GB of storage.

Memory

Memory consumption tends to increase with the complexity and volume of API requests.

APIs employing operations that require loading payloads into memory, such as encryption policies, payload transformation policies, and advanced logging functionalities, may require additional memory to accommodate the processing overhead. Similarly, high-throughput environments with a large volume of concurrent requests may necessitate increased memory allocation to ensure optimal performance and prevent resource exhaustion.

Administrators should carefully assess the memory requirements of their Gravitee APIM deployments based on factors such as anticipated API traffic patterns, payload sizes, and the specific policies implemented within each API. Regular monitoring and capacity planning efforts are essential to accurately gauge memory usage trends over time, allowing for proactive adjustments to infrastructure resources to meet evolving workload demands.

CPU

The CPU load of Gravitee APIM Gateways is directly proportional to the volume of API traffic they handle.

Monitoring CPU load serves as a crucial metric for evaluating the overall load level of the Gateways and determining the need for horizontal scalability. For instance, if the CPU utilization consistently exceeds a predefined threshold, such as 75%, it indicates that the Gateways are operating near or at capacity, potentially leading to performance degradation or service disruptions under high loads.

By regularly monitoring CPU load levels, administrators can assess the current capacity of the Gateways and make informed decisions regarding horizontal scalability. Horizontal scalability involves adding additional Gateway instances to distribute the workload and alleviate resource contention, thereby ensuring optimal performance and responsiveness for API consumers. Scaling horizontally based on CPU load enables organizations to effectively accommodate fluctuating API traffic patterns and maintain service reliability during peak usage periods.

Performance

To optimize the performance and cost-effectiveness of your APIM Gateway, consider the following factors when sizing your infrastructure:

The number of deployed APIs

Deployed APIs are maintained in memory. Increasing the number of deployed APIs consumes more memory.

The number of plugins on an API

The more plugins you add to your APIs, the more demand you place on your Gateway, which could negatively impact latency. Some plugins, such as `generate-http-signature`, are particularly CPU intensive. Others, when badly configured or handling large payloads, can require excessive memory or CPU.

Payload size

The Gateway is optimized to minimize memory consumption when serving requests and responses, so payload data is only loaded to memory when necessary. Some plugins, such as `json-xml`, `xslt`, `cache`, require that the entire payload is loaded into memory. When using these plugins, you must adjust the available memory allocated to the Gateway. We recommend using an initial value of `Maximum payload size x Maximum throughput`, which you can refine as needed.

Analytics and logging

Gravitee offers multiple methods to export analytics using reporters. Depending on throughput and the level of precision used for logging, you may need to increase the memory or disk space of your Gateway and choose the reporter best suited to handle your traffic analytics.

Rate limit and quota

Rate limit, quota, and spike arrest are patterns that are commonly applied to control API consumption. By default, Gravitee applies rate limiting in strict mode, where defined quotas are strictly respected across all load-balanced Gateways. For high throughput, we recommend using Redis, but keep in mind that some amount of CPU is required to call Redis for each API request where rate limiting is enabled.

Cryptographic operations

TLS, JWT encryption/decryption, and signature verifications can be CPU intensive. If you plan to handle high throughput that involves many costly operations, such as JWT signature, HTTP signature, or SSL, you may need to increase your CPU to keep the Gateway's latency as low as possible.

High availability

At least two Gateway instances are required to ensure your platform will experience minimum downtime in the event of critical issues or during rolling updates. In practice, you should set up the number of Gateway instances your platform requires to satisfy your performance criteria, plus one more. Then, if one instance is compromised, the remaining instances are able to handle all traffic until the failing instance recovers.

To increase resilience and uptime, you must eliminate single points of failure (SPOF), ensure reliable crossover, and detect failures as they occur.

Eliminate SPOF

One critical aspect of ensuring system reliability is the elimination of single points of failure (SPOFs). A single point of failure refers to any component within a system that, if it fails, will cause the entire system to fail. To mitigate this risk, redundancy is introduced, allowing for continued operation even if one component fails.

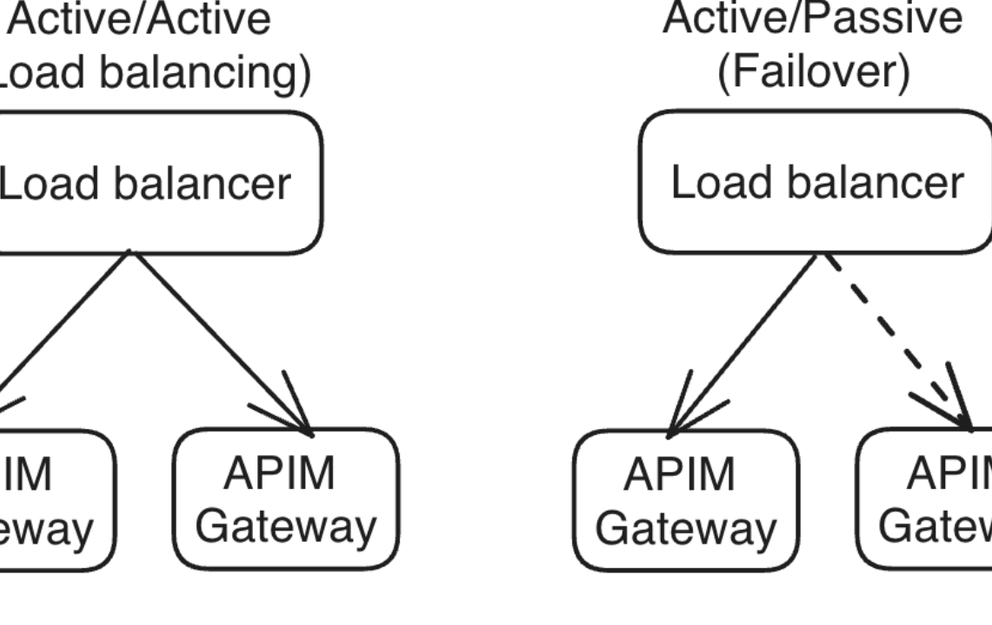
In the context of APIM, redundancy is achieved by deploying multiple instances of the APIM Gateway and optionally, Alert Engine. These instances are configured to operate in either Active/Active or Active/Passive mode, depending on the specific requirements and configurations of the system.

Active/Active Mode

In Active/Active mode, both instances of the component are actively processing requests or performing their respective functions simultaneously. This setup distributes the workload across multiple instances, thereby reducing the risk of overload on any single component. In the event of a failure in one instance, the remaining instance(s) continue to operate without interruption, ensuring continuous service availability.

Active/Passive Mode

Alternatively, Active/Passive mode involves designating one instance as active while the other remains in standby mode, ready to take over operations if the active instance fails. In this setup, the passive instance remains idle until it is needed, thereby conserving resources. Automatic failover mechanisms are employed to detect failures in the active instance and seamlessly transition operations to the passive instance without causing service disruptions.



Load balancer

> **VM installation**
> Use dedicated VMs for the Gateway and Alert Engine instances.

Reliable crossover

To ensure seamless and reliable traffic distribution to the Gravitee API Gateways, it is essential to implement a robust load-balancing solution (e.g., Nginx, HAProxy, F5, Traefik, Squid, Kemp, LinuxHA, etc.). By placing a reliable load balancer in front of the Gateways, incoming requests can be efficiently distributed across multiple Gateway instances, thereby optimizing performance and enhancing system reliability.

Health Checks

Incorporating active or passive health checks into the load balancer configuration is essential for maintaining the reliability of the crossover setup. Health checks monitor the status and availability of backend Gateway instances, enabling the load balancer to make informed routing decisions and dynamically adjust traffic distribution based on the health and performance of each instance.

- **Active Health Checks**: Active health checks involve sending periodic probes or requests to the backend instances to assess their health and responsiveness. If an instance fails to respond within a specified timeout period or returns an error status, it is marked as unhealthy, and traffic is diverted away from it until it recovers.
- **Passive Health Checks**: Passive health checks rely on monitoring the actual traffic and responses from the backend instances. The load balancer analyzes the responses received from each instance and detects anomalies or errors indicative of a failure. Passive health checks are typically less intrusive than active checks but may have slightly longer detection times.

There are some key differences to note between active and passive health checks as noted in the table below:

	Active health checks	Passive health checks (circuit breakers)
Re-enable a backend	Automatically re-enables a backend in the backend group as soon as it is healthy	Cannot automatically re-enable a backend in the backend group as soon as it is healthy
Additional traffic	Produces additional traffic to the target	Does not produce additional traffic to the target
Probe endpoint	Requires a known URL with a reliable status response in the backend to be configured as a request endpoint (e.g., "/"). By providing a custom probe endpoint for an active health checker, a backend can determine its health metrics and produce a status code to be consumed by Gravitee. Even though a target continues to serve traffic which looks healthy to the passive health checker, it can respond to the active probe with a failure status, essentially requesting to stop taking new traffic.	Does not require configuration

Detect failures

Monitoring the health and performance of Gravitee APIM Gateways and Management API is crucial for ensuring optimal operation and identifying potential issues before they impact users. By actively monitoring various metrics and endpoints, administrators can proactively address any anomalies and maintain the reliability of the API infrastructure.

Gateway Internal API Endpoints

The Gateway internal API and Management API Internal API provide a set of RESTful endpoints that enable administrators to retrieve vital information about the node status, configuration, health, and monitoring data.

Mock Policy for Active Health Checks

Utilizing an API with a Mock policy enables administrators to perform active health checks on the Gravitee APIM Gateways. By configuring mock endpoints that simulate various scenarios, such as successful requests, timeouts, or errors, administrators can verify the Gateway's responsiveness and behavior under different conditions.

Prometheus Metrics

Integration with Prometheus allows administrators to expose and collect metrics related to Gravitee APIM Gateways, including Vert.x 4 metrics. By accessing the `/_node/metrics/prometheus` endpoint on the internal API, administrators can retrieve detailed metrics with customizable labels, enabling them to monitor system performance and identify trends over time.

OpenTracing with Jaeger

Enabling OpenTracing with Jaeger facilitates comprehensive tracing of every request that passes through the API Gateway. This tracing capability offers deep insights into the execution path of API policies, enabling administrators to debug issues, analyze performance bottlenecks, and optimize API workflows effectively.